

SVM

Veri Mad. - Hafta 10

HAFTA 3: Makine Öğreniminde Zarafet ve Güç

Bu hafta, teorik olarak en zarif ve pratik olarak en güçlü sınıflandırma algoritmalarından biri olan Destek Vektör Makineleri'ni (Support Vector Machines - SVM) inceliyoruz.

SVM, özellikle yüksek boyutlu ve doğrusal olarak ayıramayan karmaşık veri setlerinde üstün performans göstermesiyle bilinir.

Sunum, üniversite düzeyinde teorik temellerden, el yazısı rakam tanıma uygulamasına kadar derinlemesine analiz sunacaktır.



Haftanın Gündemi: Dört Ana Başlık

1. Algoritmanın Teorik Temelleri: Maksimum Marj, Destek Vektörleri, Yumuşak Marj ve Kernel Hilesi.
2. Haftanın Veri Seti: El Yazısı Rakamlar (Digits Dataset) ve Zorlukları.
3. Google Colab Uygulaması: Python ve Scikit-learn ile adım adım model kurulumu.
4. Model Çıktılarının Değerlendirilmesi: Hata analizi ve performans ölçütleri.

Neden SVM Öğrenmeliyiz?

Daha önce incelenen Lojistik Regresyon, sınıfları ayıracak rastgele bir sınır çizgisi bulmaya odaklanır.

SVM ise çok daha iddialı bir hedef belirler: Veri noktaları arasındaki ayrımı maksimize etmek.

SVM, özellikle sınırlı örneklem sayısı ve yüksek boyutluluk içeren problemlerde sağlam (robust) sonuçlar üretir.



I. Teorik Temeller: Maksimum Marj Kavramı

Maksimum Marj (Maximum Margin), SVM felsefesinin kalbidir.

SVM, iki sınıfın veri noktaları arasındaki 'caddeyi' veya 'güvenlik boşluğunu' (marjı) maksimum yapan en uygun hiper-düzlemi (hyperplane) bulmayı amaçlar.

Bu maksimize edilmiş 'güvenlik boşluğu', modelin genelleme (test verilerinde başarılı olma) yeteneğini doğrudan artırır.



Hiper-düzlem Nedir?

2 Boyutlu uzayda (2 feature) bir hiper-düzlem, iki sınıfı ayıran bir çizgi (line) olarak düşünülebilir.

3 Boyutlu uzayda bir hiper-düzlem, iki sınıfı ayıran bir düzlem (plane) olarak düşünülebilir.

Genel olarak, N boyutlu uzayda, sınıfları ayıran N-1 boyutlu bir yüzeydir.

SVM'in amacı, sadece bir ayırım yüzeyi bulmak değil, bu yüzeyin sınıflara olan mesafesini maksimize etmektir (Maksimum Marj).



Marjın Matematiksel Anlamı

Marj, hiper-düzlem ile en yakın veri noktaları arasındaki mesafenin iki katıdır. Model, en yakın noktalara olan mesafeyi maksimize ederek, sınıflar arasındaki en net ve 'en güvenli' ayrımı sağlar. Büyük bir marj, yeni, görülmemiş veri noktalarının doğru tarafa sınıflandırılma olasılığını artırır (daha iyi genelleme).



Hard Margin Sınıflandırma

Eğer iki sınıf, hiçbir hata yapılmadan bir hiper-düzlem ile tamamen ayrılabilirse, bu duruma 'Hard Margin' (Sert Marj) denir.

Bu durumda, tüm veri noktaları marjın dışındadır veya tam olarak marjın kenarındadır.

Hard Margin, pratik gerçek dünya veri setlerinde (özellikle gürültülü verilerde) nadiren uygulanabilir.

Model aşırı öğrenmeye (overfitting) eğilimli olabilir eğer veri gürültülü ise.



II. Destek Vektörleri: Kritik Noktalar

Destek Vektörleri (Support Vectors), marjın kenarlarına tam olarak değen veya 'cadde' (marj) içine giren veri noktalarıdır.

Modelin matematiksel tanımı (yani hiper-düzlemin konumu ve eğimi), yalnızca bu kritik noktalar tarafından belirlenir.

Diğer tüm veri noktaları (marjın dışında kalanlar) silinse bile, hiper-düzlemin konumu değişmez.



SVM'in Verimlilik Avantajı

Destek Vektörleri, SVM'in hafıza açısından verimli olmasını sağlayan temel özelliktir.

Eğitim tamamlandıktan sonra, modeli saklamak için tüm veri setine ihtiyaç duyulmaz.

Sadece Destek Vektörleri ve onlara karşılık gelen ağırlıklar saklanır. Bu, çok büyük veri setleriyle çalışırken ciddi bir avantaj sağlar.



Destek Vektörleri ve Aşırı Öğrenme

Destek Vektörleri, kararsız bölgelerin sınırındaki en zor örnekleri temsil eder. Modelin sadece bu zor örnekler üzerinden eğitilmesi, gereksiz gürültüden etkilenmeyi azaltır. Böylece SVM, genelleme yeteneğini artırarak aşırı öğrenme riskini minimize eder.



III. Yumuşak Marj (Soft Margin) İhtiyacı

Gerçek dünyadaki veri setleri genellikle gürültülü veya karışıktır ve nadiren mükemmel bir şekilde doğrusal olarak ayrılabilir (Hard Margin). Yumuşak Marj (Soft Margin) kavramı, modelin bazı veri noktalarının marjı ihlal etmesine (yanlış tarafta olmasına veya marjın içine girmesine) izin verir. Bu esneklik, modelin aykırı değerlere (outliers) karşı daha sağlam olmasını sağlar.

Marj ihlali ve Cezalandırma

Yumuşak Marj, 'kayıp' (loss) fonksiyonuna bir ceza (penalty) terimi eklenerek uygulanır.

Bu ceza, bir veri noktasının marjı ne kadar ihlal ettiğini ölçer (slack variables – sarkaç değişkenleri).

Amaç: Hem marjı maksimize etmek hem de ihlal cezasını minimize etmektir.



C Hiperparametresi: Düzenleme (Regularization)

Yumuşak Marjin esnekliği, C (Regularization parameter) hiperparametresi tarafından kontrol edilir.

C değeri, 'büyük marj arzusu' ile 'ihlalleri cezalandırma' arasındaki dengeyi ayarlar.

Bu, bir dengeleme mekanizmasıdır: Marjı maksimize etmeye karşı hataları minimize etme.



Düşük C Değeri Ne Anlama Gelir?

Düşük C değeri (örn. $C=0.01$), hata toleransının yüksek olduğu anlamına gelir. Model, ihlalleri (yanlış sınıflandırmaları) daha az cezalandırır. Sonuç: Daha geniş bir marj elde edilir (daha fazla hataya izin veren bir cadde). Bu, potansiyel olarak az öğrenmeye (underfitting) yol açabilir, ancak aykırı değerlere karşı çok sağlamdır.

Yüksek C Değeri Ne Anlama Gelir?

Yüksek C değeri (örn. $C=100$), hata toleransının düşük olduğu anlamına gelir. Model, ihlalleri çok şiddetli cezalandırır ve neredeyse mükemmel bir ayırım ister.

Sonuç: Daha dar bir marj elde edilir (daha az hataya izin veren bir cadde). Bu, eğitim verisine çok sıkı uyum sağlayarak potansiyel aşırı öğrenmeye (overfitting) yol açabilir.

C Parametresi Seçimi Özeti

Düşük C: Daha sağlam (robust) model, yüksek bias, düşük variance.

Yüksek C: Daha karmaşık model, düşük bias, yüksek variance (aşırı öğrenme riski).

Optimal C değeri, çapraz doğrulama (cross-validation) yöntemleri ile belirlenmelidir.

C'nin ayarlanması, modelin basitliği ile eğitim verisine uyumu arasındaki dengeyi sağlar.



IV. Kernel Trick (Çekirdek Hilesi)

Kernel Trick, SVM'in doğrusal olarak ayıramayan karmaşık veri setleriyle başa çıkmasını sağlayan sihirli bir yöntemdir.

Temel sorun: Eğer veri, mevcut boyutunda (örneğin 2D'de bir daire şeklinde) bir çizgi ile ayıramıyorsa ne olur?

Çözüm: Veriyi daha yüksek bir boyuta taşıyarak orada doğrusal ayırım bulmak.

Boyut Dönüşümünün Mantığı

Kernel fonksiyonu, veriyi fiziksel olarak yüksek boyuta taşımadan, bu yüksek boyutta iç çarpımların (dot products) sonucunu hesaplar. Bu 'örtük haritalama', hesaplama maliyetini çok büyük ölçüde düşürür. Yüksek boyutlu uzayda, orijinal uzayda doğrusal olmayan karar sınırı, doğrusal bir hiper-düzlem ile temsil edilebilir.

En Yaygın Kullanılan Çekirdekler – LINEAR

Linear Kernel: $K(x_i, x_j) = x_i \cdot x_j$.

Bu çekirdek, veriyi dönüştürmez (hiçbir dönüşüm yapmaz).

Eğitim verisinin zaten doğrusal olarak iyi ayrılabilir olduğu durumlarda kullanılır.

Hesaplama açısından hızlıdır ve çok sayıda özellik (feature) içeren seyrek veri setlerinde tercih edilir.



En Yaygın Kullanılan Çekirdekler – POLYNOMIAL

Polynomial (Poly) Kernel: $K(x_i, x_j) = (\alpha x_i \cdot x_j + r)^d$.

Veriyi d. dereceden bir polinomsal uzaya dönüştürür.

d (derece) hiperparametresi model karmaşıklığını belirler.

Çok yüksek d değerleri aşırı öğrenmeye yol açabilir ve genellikle RBF kadar popüler değildir.



En Güçlü Çekirdek – RBF (Radial Basis Function)

RBF (Radial Basis Function) veya Gauss Çekirdeği, en yaygın kullanılan ve en güçlü çekirdektir.

Teorik olarak, veriyi sonsuz boyutlu bir uzaya haritalayabilir.

Doğrusal olmayan, karmaşık ve iç içe geçmiş veri setlerinde mükemmel sonuçlar verir.

El yazısı rakam tanıma gibi yüksek boyutlu problemler için idealdir.



RBF Çekirdeği: Gamma Hiperparametresi

RBF çekirdeği, 'gamma' hiperparametresi ile kontrol edilir. Gamma, tek bir veri noktasının (veya destek vektörünün) karar sınırını ne kadar uzaktan etkileyebileceğini belirler. Yüksek Gamma: Sadece çok yakın noktalar etkiler. Dar, karmaşık karar sınırları oluşur (aşırı öğrenme riski).

RBF: Düşük Gamma Değeri

Düşük Gamma değeri, destek vektörlerinin etkisinin daha geniş bir alana yayıldığı anlamına gelir.

Bu, daha yumuşak ve daha genelleştirilebilir karar sınırları oluşturur.

Model, tüm veri setini dikkate alarak daha küresel bir perspektifle ayırım yapar. Genellikle 'gamma='scale' otomatik seçeneği ilk denemelerde iyi sonuç verir.

RBF: Yüksek Gamma Değeri

Yüksek Gamma değeri, modelin sadece çok yakınındaki veri noktalarından etkilenmesi anlamına gelir.

Bu, karar sınırının her bir veri noktasına çok yakından uyum sağlamasına neden olur.

Sonuç: Yüksek varyans ve aşırı öğrenme eğilimi gösteren, çok dalgalı ve lokalize karar sınırlarıdır.

V. Haftanın Veri Seti: El Yazısı Rakamlar

Haftanın pratik uygulaması için El Yazısı Rakamlar (Digits Dataset) veri setini kullanıyoruz.

Bu veri seti, popüler makine öğrenimi kütüphanesi scikit-learn içinde yerleşik olarak bulunur.

İçeriği: El yazısı rakamların (0'dan 9'a kadar) dijitalleştirilmiş görüntüleri.

Digits Dataset'in Yapısı ve Boyutları

Çözünürlük: Her görüntü **8x8 piksel** çözünürlüğe sahiptir.

Örneklem Sayısı: Veri seti yaklaşık **1797** örneklem içerir.

Özellik Sayısı (Feature): $8 \times 8 = 64$ piksel, yani veri seti **64 özelliğe** sahiptir (yüksek boyutluluk).

Hedef: 10 sınıflı (0, 1, ..., 9) bir sınıflandırma problemidir.

MNIST ile Karşılaştırma

Digits veri seti, genellikle çok daha büyük ve karmaşık olan ****MNIST**** veri setinin (28x28 piksel) daha küçük ve yönetilebilir bir versiyonu olarak kullanılır. Bu, SVM gibi yüksek boyutluluk algoritmalarının temel prensiplerini göstermek için pedagojik olarak idealdir.

Pedagojik Önem: Yüksek Boyutluluk

Veri setinin 64 boyutlu olması, bu hafta için kritik öneme sahiptir. Bu, bir görüntüdeki her pikselin, uzayda bir boyut olarak hareket ettiği anlamına gelir. Bu kadar yüksek boyutlu uzayda etkili ayırım yüzeyleri bulmak zordur.

Boyutluluk Laneti (Curse of Dimensionality)

Hafta 2'de öğrendiğimiz K-En Yakın Komşu (K-NN) gibi algoritmalar, yüksek boyutlu verilerde 'Boyutluluk Laneti' nedeniyle zorlanırlar. Veri seyrelir ve mesafe ölçümleri anlamını yitirir. SVM'in (özellikle Kernel Trick ile) bu lanete karşı nasıl koyduğunu göstermek için Digits veri seti mükemmeldir.

SVM'in Yüksek Boyutluluk Uzmanlığı

SVM (özellikle RBF çekirdeği kullanıldığında), bu tür yüksek boyutlu uzaylarda bile sınıflar arasında etkili ve sağlam (robust) ayırım yüzeyleri bulabilme yeteneğine sahiptir.

Kernel Trick sayesinde, 64 boyutlu uzaydan daha da yüksek boyutlara haritalama yaparak doğrusal ayrımı mümkün kılar.



VI. Google Colab Uygulaması: Verinin Yüklenmesi

Uygulamanın ilk adımı, gerekli kütüphaneleri ve veri setini içeri aktarmaktır. scikit-learn kütüphanesi içindeki 'sklearn.datasets' modülü kullanılır. 'load_digits()' fonksiyonu, veri setini kolayca yüklememizi sağlar. Yüklenen veri, özellik matrisi (X) ve hedef vektörü (y) olarak ayrılır.

Veri Ön İşlemesi: Piksel Aralığı Analizi

Digits veri setindeki piksel değerleri 0 ile 16 arasında değişmektedir. 0, siyah rengi (arka planı) temsil ederken, 16 en yüksek beyaz/gri tonunu temsil eder. Bu aralık, birçok makine öğrenimi algoritması ve özellikle RBF çekirdeği için uygun olmayabilir.



Veri Ön İşlemesi: Ölçeklendirme (Scaling)

RBF çekirdeğinin ve optimizasyon sürecinin performansını artırmak için, verinin ölçeklendirilmesi şiddetle tavsiye edilir.

'StandardScaler' ile veriyi ölçeklendirerek, her özelliğin ortalamasını 0 ve standart sapmasını 1 yaparız (Z-Skor normalizasyonu).

Ölçekleme, SVM'in uzaklık tabanlı optimizasyonunun doğru çalışması için kritiktir.

Veri Ayırma: Eğitim ve Test Setleri

Modelin görülmemiş veriler üzerindeki genelleme yeteneğini ölçmek için veri seti bölünmelidir.

'train_test_split' fonksiyonu kullanılarak, özellik matrisi (X) ve hedef vektörü (y) eğitim (train) ve test (test) alt kümelerine ayrılır.

Tipik olarak %70 eğitim, %30 test veya %80 eğitim, %20 test oranları kullanılır.

VII. Modelin Kurulması ve Eğitimi

SVM uygulaması için 'sklearn.svm' modülü içinden 'SVC' (Support Vector Classifier) sınıfı çağrılır.

Bu uygulamada, iki farklı çekirdek (kernel) kullanılarak karşılaştırma yapılacaktır.

Çekirdek seçimi, veri setinin doğasına bağlıdır (doğrusal mı, doğrusal olmayan mı?)

Model Eğitimi 1: Doğrusal (Linear) Çekirdek

İlk model, doğrusal olarak ayrılabilirliği test etmek için Linear çekirdek ile kurulur:

```
'clf_linear = SVC(kernel='linear')'
```

Bu model, veriyi dönüştürmeden 64 boyutlu uzayda doğrusal bir hiper-düzlem arar.

Model Eğitimi 2: RBF Çekirdek

İkinci model, problemdeki karmaşıklığı ele almak için RBF çekirdek ile kurulur:
'clf_rbf = SVC(kernel='rbf', C=1.0, gamma='scale')'.
'C=1.0' standart bir başlangıç değeri olarak kullanılır.
'gamma='scale', gamma değerinin otomatik olarak $(1 / (n_features * X.var()))$ olarak hesaplanmasını sağlar.



Modelin Eğitilmesi (Fit İşlemi)

Her iki model de ölçeklenmiş eğitim verisi kullanılarak eğitilir:
'fit(X_train_scaled, y_train)'.

Bu eğitim aşamasında, optimizasyon algoritması maksimum marjı sağlayacak destek vektörlerini ve hiper-düzlem parametrelerini bulur. Bu işlem, özellikle RBF çekirdeği için çekirdek matrisini hesaplamayı içerir.

Tahmin Aşaması (Prediction)

Eğitilmiş model, daha önce hiç görmediği test verileri üzerinde tahmin yapmak için kullanılır.

Ölçeklenmiş test verileri modele beslenir:

```
'y_pred = clf_rbf.predict(X_test_scaled)'
```

Bu adımda, test noktalarının hangi tarafa düştüğü (hangi sınıfa ait olduğu) hesaplanır.



VIII. Model Çıktılarının Görselleştirilmesi

Problemimiz 10 sınıflı (multi-class) ve 64 boyutludur. İnsan gözü 2 veya 3 boyutu algılayabildiğinden, 64 boyutlu uzaydaki karar sınırlarını çizmek veya görselleştirmek fiziksel olarak imkansızdır. Bu nedenle, modelin davranışını anlamak için farklı görselleştirme ve analiz tekniklerine ihtiyaç duyarız.



Hata Analizi (Error Analysis) Yaklaşımı

Karar sınırlarını çizmek yerine, modelin nerede yanlış tahmin yaptığını incelemek çok daha öğreticidir.

Hata analizi, bir makine öğrenimi modelini iyileştirmenin temel yollarından biridir.

Amacımız: Modelin hatalı sınıflandırdığı örneklerin görsel özelliklerini inceleyerek problemin zorluğunu anlamaktır.

Hatalı Tahminlerin Tespiti

Test setindeki model tahminleri ('y_pred') ile gerçek etiketler ('y_test') karşılaştırılır.

Karşılaştırma sonucunda, 'y_pred != y_test' koşulunu sağlayan örneklerin indisleri (indeksleri) bulunur.

Bu indisler, hatalı sınıflandırılan görüntülerin ham verisini çekmek için kullanılır.

Görüntülerin Yeniden Şekillendirilmesi

Ham veri, 64 piksellik tek boyutlu bir dizidir (feature vektör).
Görüntüyü görselleştirebilmek için bu dizinin 8x8'lik iki boyutlu bir matris olarak yeniden şekillendirilmesi gerekir.
Bu dönüşüm, görüntünün piksel yoğunluk haritasını oluşturur.



Görselleştirme İşlemi

Python'da 'matplotlib.pyplot.subplots' kütüphanesi kullanılarak bir ızgara yapısı oluşturulur.

Hatalı tahmin edilen her bir örnek, 'plt.imshow' fonksiyonu ile ızgaradaki bir alt grafiğe (subplot) yerleştirilir.

'imshow', piksel değerlerine göre yoğunluk haritası (genellikle gri tonlama) oluşturur.

Görsel Çıktının İncelenmesi

Her bir alt grafiğin başlığına, modelin yaptığı tahmin ve gerçek etiket yazılır (örn: 'Tahmin: 9, Gerçek: 4').

Bu görselleştirme, modelin neden hata yaptığını insani bir bakış açısıyla gösterir.

Örneğin, '9' rakamının '4'e çok benzemesi veya el yazısının çok kötü olması hata nedenlerinden bazılarıdır.



IX. Performans Değerlendirmesi: Genel Bakış

Digits veri seti, 10 farklı sınıf (0'dan 9'a) içerdiği için çok sınıflı bir problemdir. Performans değerlendirme, sadece genel doğruluk (accuracy) değil, her bir sınıf üzerindeki performansı da detaylıca analiz etmelidir. Bu analiz için Hata Matrisi ve Sınıflandırma Raporu kullanılır.



Hata Matrisi (Confusion Matrix) Tanımı

Hata Matrisi, tahmin edilen sınıflar ile gerçek sınıfların sayısını gösteren bir tablodur.

10 sınıflı problemimizde bu matris 10x10 boyutunda olacaktır.

Ana çapraz (diagonal) üzerindeki değerler doğru tahminleri, çapraz dışındaki değerler ise yanlış tahminleri (karışıklıkları) gösterir.



Hata Matrisinin Görselleştirilmesi

Hata matrisi, genellikle 'seaborn.heatmap()' fonksiyonu kullanılarak renkli bir ısı haritası şeklinde görselleştirilir.

Bu görselleştirme, hangi rakam çiftlerinin model tarafından en çok karıştırıldığını net bir şekilde görmemizi sağlar.

Örneğin, 4 ile 9 veya 3 ile 8 arasındaki karışıklık yoğunlukları incelenebilir.

Sınıflandırma Raporu (Classification Report)

Sınıflandırma raporu, 0'dan 9'a kadar her bir rakam sınıfı için ayrı ayrı metrikler sunar.

Bu metrikler Precision, Recall ve F1-Score'dur.

Bu rapor, modelin hangi rakamı tanımakta çok iyi (örn. '1' veya '0') ve hangi rakamları tanımakta zorlandığını (örn. '8') detaylı bir şekilde analiz etme imkanı verir.

Precision (Kesinlik) Nedir?

Precision: Modelin bir sınıfı 'X' olarak tahmin ettiğinde, bunun gerçekten 'X' olma olasılığıdır.
(Doğru Pozitifler) / (Doğru Pozitifler + Yanlış Pozitifler).
Yüksek Precision, modelin tahminlerine güvenilebileceğimiz anlamına gelir.

Recall (Duyarlılık) Nedir?

Recall: Gerçekte 'X' sınıfına ait olan tüm örneklerden, modelin doğru bir şekilde 'X' olarak tahmin edebildiği orandır.

$(\text{Doğru Pozitifler}) / (\text{Doğru Pozitifler} + \text{Yanlış Negatifler})$.

Yüksek Recall, modelin hiçbir örneği gözden kaçırmadığını gösterir.

F1-Score: Hassasiyet ve Duyarlılık Dengesi

F1-Score, Precision ve Recall'un harmonik ortalamasıdır.
Bu metrik, dengesiz sınıf dağılımlarında veya tek bir metriğe odaklanmak istemediğimizde genel performansı özetler.
Hem yüksek Precision hem de yüksek Recall elde etmek için çabalarız.



X. Çekirdek Karşılaştırması ve Sonuç

Sınıflandırma raporunda elde edilen sonuçlar karşılaştırılmalıdır. Doğrusal (Linear) çekirdek sonuçları ile RBF çekirdeği sonuçları yan yana incelenir.

Beklenen sonuç: El yazısı rakam tanıma (doğrusal olmayan, karmaşık bir problem) için RBF çekirdeği, genellikle doğrusal çekirdeğe göre ****üstün**** performans gösterir.



RBF Üstünlüğünün Analizi

RBF çekirdeğinin üstünlüğü, veriyi sonsuz boyutlu bir uzaya haritalayarak, 64 boyutlu uzayda mümkün olmayan doğrusal olmayan karar yüzeylerini (hiper-düzlemler) bulabilmesinden kaynaklanır. Bu, modelin sınıflar arasındaki ince ve karmaşık farkları yakalamasını sağlar (örneğin 4 ile 9 arasındaki piksel farklılıkları).



Pratik Çıkarımlar ve Parametre Ayarı

Optimal SVM performansı için sadece doğru çekirdek seçimi değil, aynı zamanda C ve Γ (RBF için) hiperparametrelerinin de doğru ayarlanması gerekir.

Bu parametreler Grid Search veya Random Search gibi yöntemlerle sistematik olarak taranarak en iyi genelleme performansı elde edilir.

Yanlış parametre seçimi, az öğrenme (düşük C , düşük Γ) veya aşırı öğrenmeye (yüksek C , yüksek Γ) yol açabilir.



SVM Teorisi ve Optimizasyon

SVM'in arkasındaki optimizasyon problemi, konveks optimizasyon problemidir. Bu, bulunan çözümün (maksimum marj hiper-düzlemi) global olarak optimal olduğu anlamına gelir.

Bu özellik, lojistik regresyon gibi algoritmaların yerel minimumlara takılma riskine kıyasla SVM'e büyük bir avantaj sağlar.



Destek Vektör Makineleri Özeti I

SVM, maksimum marjı hedefleyen bir sınıflandırma algoritmasıdır. Destek Vektörleri, modelin konumunu belirleyen kritik örneklerdir ve hafıza verimliliği sağlarlar. C parametresi, marj genişliği ile sınıflandırma hatası arasındaki dengeyi ayarlar (Yumuşak Marj).



Destek Vektör Makineleri Özeti II

Kernel Trick (Çekirdek Hilesi), doğrusal olarak ayıramayan verileri yüksek boyutlu uzaya taşıyarak doğrusal ayırım bulmamızı sağlar.

RBF çekirdeği, karmaşık ve yüksek boyutlu veri setlerinde (Digits gibi) üstün performans sunar.

Digits veri seti, 64 boyutlu uzayda SVM'in güçlü genelleme yeteneğini göstermek için ideal bir pedagojik araçtır.



Uygulama Özeti ve Öğrenilenler

Veri ön işleme (ölçekleme), SVM performansı için kritik öneme sahiptir. 64 boyutlu problemde karar sınırlarını çizmek yerine, Hata Matrisi ve Hata Analizi ile performans değerlendirilmesi önemlidir. Çok sınıflı problemlerde Precision, Recall ve F1-Score gibi sınıf bazlı metrikler gereklidir.



Tek Sınıflı SVM (One-Class SVM): Aykırı değer tespiti için SVM kullanımı.
SVM Regresyon (SVR): Regresyon problemleri için SVM'in adaptasyonu.
Çekirdek Seçimi Problemi: En uygun çekirdeği otomatik olarak seçme yöntemleri (örneğin, Çekirdek Öğrenme - Kernel Learning).

Teşekkürler

Sorularınız ve katkılarınız için teşekkür ederiz.
Bir sonraki hafta, makine öğrenimindeki yeni bir konseptle devam edeceğiz.



SVM

Süleyman **EZDEMİR**

suleyman.ezdemir@giresun.edu.tr

